

autonomy-light 튜닝 명세서

작성일: 2026-07-07

수정일: 2026-07-08

소스 기본값 파일: config/autonomy_light.yaml

Jetson 런타임 설정 파일: /etc/cocelo/autonomy-light/autonomy_light.yaml

목적: MID360 + Point-LIO 기반 autonomy-light 에서 제어용 height map 품질, ROS 2 도메인 분리, 장애물 반응성, 노이즈 억제를 현장에서 조정하기 위한 파라미터 설명서.

0. 적용 방법

.deb 로 설치된 Jetson에서 실제 운용 값을 바꾸려면 아래 파일을 수정한다.

```
sudo nano /etc/cocelo/autonomy-light/autonomy_light.yaml
```

파라미터는 실행 시작 시 읽힌다. 실행 중 YAML을 바뀌도 자동 반영되지 않으므로 기존 프로세스를 종료한 뒤 다시 실행한다.

```
# 실행 중인 terminal에서 Ctrl-C 후
autonomy-light --real
```

남아 있는 프로세스가 의심되면 정리 후 실행한다.

```
sudo pkill -f autonomy-light || true
sudo pkill -f autonomy_light || true
autonomy-light --real
```

새 .deb 의 기본값을 바꾸려면 repo의 config/autonomy_light.yaml 을 수정한 뒤 Jetson에서 다시 packaging한다. 이미 설치된 장비의 현장 값은 /etc/cocelo/autonomy-light/autonomy_light.yaml 이 우선이다.

튜닝 후 최소 확인:

```
ROS_DOMAIN_ID=0 ros2 topic hz /autonomy_light/height_map_data
ROS_DOMAIN_ID=0 ros2 topic echo /autonomy_light/height_map_data --once
ROS_DOMAIN_ID=0 ros2 topic hz /autonomy_light/odom
```

내부 Point-LIO 입력이 살아 있는지도 함께 본다.

```
ROS_DOMAIN_ID=42 ros2 topic hz /aft_mapped_to_init
ROS_DOMAIN_ID=42 ros2 topic hz /point_lio/local_map
```

1. 튜닝 원칙

- 제어용 PC에는 무거운 perception topic을 보내지 않는다.
 - internal_ros_domain_id 는 LiDAR, Point-LIO, local map 입력용이다.
 - external_ros_domain_id 는 제어용 출력만 보내는 domain이다.
 - debug_local_map_ros_domain_id 를 외부 domain과 같게 두면 /point_lio/local_map 이 제어용 PC로 흘러가 렉을 만들 수 있다.
- local map 자체가 두꺼워도 height map은 제어에 필요한 표면만 선택한다.
 - 현재 기본 backend는 autonomy_min_z 이다.
 - 단순 min-z는 낮은 outlier에 약하므로 supported min-z와 obstacle override를 같이 사용한다.
- 노이즈 억제와 장애물 반응성은 trade-off다.
 - 노이즈를 줄이려면 min_points_per_cell, min_support_neighbors를 올린다.
 - 장애물 반응을 빠르게 하려면 obstacle_min_height, obstacle_min_points를 낮춘다.
 - 장애물 잔상을 줄이려면 previous grid carry-over, interpolation, hole fill을 줄인다.
- 한 번에 하나의 계열만 바꾼다.
 - geometry, height origin, min-z, filter/fill을 동시에 바꾸면 원인 추적이 어렵다.
 - 바꾼 값, 주행 환경, 관찰 결과를 같이 기록한다.
- 출력 메시지 값은 raw z 높이가 아니라 distance-style 값이다.
 - /autonomy_light/height_map_data 는 data[index] = base_height - grid_z 로 발행된다.
 - 기본 base_height 는 algorithm.clipping.max_z: 0.48 이다.
 - 평지는 대체로 0.48에 가깝고, 높은 장애물일수록 값이 0.0 쪽으로 작아진다.

2. 빠른 현장 체크

제어용 PC가 렉 걸릴 때

확인:

```
ROS_DOMAIN_ID=0 ros2 topic list | grep point_lio/local_map
```

정상: 아무것도 나오지 않아야 한다.

관련 설정:

```
internal_ros_domain_id: 42
external_ros_domain_id: 0
debug_local_map_ros_domain_id: 42
```

local map을 RViz에서 보고 싶을 때

```
ROS_DOMAIN_ID=42 rviz2
```

/point_lio/local_map 은 QoS를 Best Effort로 보는 것이 안전하다.

height map이 지지직거릴 때

우선순위:

1. algorithm.min_z.min_points_per_cell 을 3에서 4로 올린다.
2. algorithm.min_z.support_band 를 0.04에서 0.05로 올린다.
3. algorithm.isolated_filter.min_support_neighbors 를 3에서 4로 올린다.
4. 마지막으로 algorithm.bilateral.passes 를 1로 켜다.

장애물 반응이 늦을 때

우선순위:

1. algorithm.min_z.obstacle_min_height 를 0.06에서 0.04로 낮춘다.
2. algorithm.min_z.obstacle_min_points 를 2에서 1로 낮춘다.
3. algorithm.isolated_filter.min_support_neighbors 를 3에서 2로 낮춘다.

장애물 잔상이 남을 때

우선순위:

1. algorithm.frame_aggregation.temporal_alpha 가 1.0인지 확인한다.
2. algorithm.hole_fill.radius 또는 interpolation_max_passes 를 줄인다.
3. 필요하다면 previous grid carry-over를 끄는 옵션을 별도로 추가한다.

grid 크기나 해상도를 바꿨을 때

확인:

```
ROS_DOMAIN_ID=0 ros2 topic echo /autonomy_light/height_map_data --once
```

resolution, x_length, y_length, data 길이가 기대와 맞아야 한다.

```
width = ceil(x_length / resolution)
height = ceil(y_length / resolution)
data.size() == width * height
```

3. 좌표계 및 센서 장착

target_frame

제어 기준 로봇 frame이다. 현재값: base_link .

- height map은 최종적으로 이 frame 기준 로봇 주변 grid로 해석된다.
- 제어용 SBC가 기대하는 기준 frame과 맞아야 한다.

height_map_frame

height map을 발행할 frame이다. 현재값: base_link_gravity .

- roll/pitch를 제거하고 yaw만 남긴 중력 정렬 frame으로 쓰는 것이 목적이다.
- 바닥이 로봇 roll/pitch에 따라 흔들리는 문제를 줄인다.

lidar_frame

LiDAR frame 이름이다. 현재값: lidar_link .

- target_to_lidar_xyz , target_to_lidar_rpy 와 함께 TF를 구성한다.
- URDF의 LiDAR frame 이름과 일관되어야 한다.

target_to_lidar_xyz

target_frame 에서 lidar_frame 까지의 translation [x, y, z] 이다. 단위는 meter.

- 값이 틀리면 local map을 height map으로 자를 때 위치가 밀린다.
- 특히 z가 틀리면 바닥 높이와 장애물 높이가 같이 어긋난다.

target_to_lidar_rpy

target_frame 에서 lidar_frame 까지의 rotation [roll, pitch, yaw] 이다. 단위는 radian.

- MID360이 뒤집혀 장착되었으면 roll 보정이 중요하다.
- 장착 방향이 틀리면 바닥이 기울거나 장애물이 이상한 방향으로 퍼진다.

4. Height Map Geometry

elevation_resolution

height map cell 크기다. 단위는 meter. 현재값: 0.05 .

- 작게 하면 장애물 edge가 날카롭지만 point support가 부족해져 노이즈와 구멍이 늘 수 있다.
- 크게 하면 안정적이지만 장애물 반응이 둔해지고 edge가 뭉툭해진다.
- 추천 범위: 0.04 ~ 0.10 .

elevation_x_length , elevation_y_length

로봇 중심 height map 크기다. 단위는 meter. 현재값: 1.2 x 1.2 .

- 크면 넓게 보지만 계산량과 외부 publish payload가 늘어난다.
- 제어에 필요한 범위만 유지하는 것이 좋다.
- 길이를 키우면 Point-LIO local map crop 범위도 같이 커져 내부 계산량이 늘 수 있다.

elevation_x_center , elevation_y_center

height map 중심을 target_frame 기준으로 얼마나 이동할지 정한다. config에 없으면 코드 기본값 0.0, 0.0 을 사용한다.

- 전방만 더 보고 싶으면 elevation_x_center 를 양수로 옮기는 방식이 가능하다.
- 중심을 옮기면 제어가 배열을 해석하는 원점도 같은 규칙으로 맞춰야 한다.

elevation_min_z , elevation_max_z

config에 없으면 코드 기본값을 사용한다. height map 후보 point를 받을 z 범위다.

- 너무 넓으면 천장, 벽, 비정상 누적점이 들어올 수 있다.
- 너무 좁으면 실제 장애물이 잘린다.
- 최종 출력 clamp는 algorithm.clipping.* 에서 따로 한다. 이 값은 후보 point filtering 범위에 가깝다.

5. Height Origin

height_origin.mode

height map의 z 기준을 정하는 방법이다. 현재값: local_floor .

- local_floor : 주변 바닥 후보를 찾아 z 기준으로 쓴다. 로봇 z/roll/pitch jitter에 강하다.
- odom : Point-LIO odom z를 그대로 쓴다.
- fixed : fixed_z 를 고정 기준으로 쓴다.

height_origin.fixed_z

mode: fixed 일 때 사용할 z 기준값이다.

height_origin.filter_alpha

height origin z의 low-pass filter 계수다. 현재값: 0.25 .

- 크게 하면 기준 높이가 빠르게 따라가지만 jitter도 커질 수 있다.
- 작게 하면 안정적이지만 stand-up 같은 큰 변화에 늦다.

height_origin.max_step

한 frame에서 height origin이 변할 수 있는 최대 z 변화량이다. 단위는 meter. 현재값: 0.03 .

- 값이 작으면 갑작스러운 z jump를 막지만 큰 자세 변화에 늦게 따라간다.
- 값이 크면 빠르지만 바닥이 출렁일 수 있다.

height_origin.floor_radius

local floor를 찾을 때 로봇 주변에서 볼 반경이다. 단위는 meter. 현재값: 0.6 .

height_origin.floor_percentile

바닥 후보 z 중 사용할 percentile이다. 현재값: 0.20 .

- 낮추면 더 낮은 바닥 표면을 잡는다.
- 올리면 낮은 outlier에 덜 민감하다.

height_origin.floor_min_points

local floor 추정에 필요한 최소 point 수다. 현재값: 20 .

- 부족하면 odom z fallback이 발생한다.

6. ROS 2 Domain 및 Topic

internal_ros_domain_id

LiDAR, Point-LIO, local map 등 heavy perception 내부 domain이다. 현재값: 42 .

external_ros_domain_id

제어용 SBC가 보는 출력 domain이다. 현재값: 0 .

- 이 domain에는 /autonomy_light/odom , /autonomy_light/height_map , /tf , /tf_static , /path 정도만 남기는 것이 좋다.

debug_local_map_ros_domain_id

/point_lio/local_map debug republish domain이다. 현재값: 42 .

- 42 : 내부에서만 local map 확인. 제어용 PC 부하가 작다.
- 0 : 외부 제어망에도 local map이 흘러간다. RViz 확인은 편하지만 렉 원인이 될 수 있다.

debug_local_map_topic

debug republish topic 이름이다. 현재값: /point_lio/local_map .

raw_lidar_topic , raw_imu_topic

Livox driver raw topic이다.

point_lio_odom_topic

Point-LIO odometry 입력 topic이다. 현재값: /aft_mapped_to_init .

point_lio_map_topic

height map 입력으로 쓰는 Point-LIO local map topic이다. 현재값: /point_lio/local_map .

point_lio_registered_topic

등록된 현재 scan topic이다. 현재값: /cloud_registered .

- cloud_registered_fill.enabled 가 true일 때 보조 fill에 사용된다.

odom_output_topic , height_map_topic , path_output_topic

외부 domain으로 내보내는 제어용 출력 topic이다.

height_map_msg_topic

custom autonomy_light/msg/HeightMap 출력 topic이다. 현재값: /autonomy_light/height_map_data .

- 제어 프로그램은 보통 이 topic을 구독한다.
- topic 이름을 바꾸면 수신 프로그램과 문서의 subscribe topic도 같이 바뀌어야 한다.

heartbeat_topic

runtime 상태 문자열 topic이다. 현재 기본값: /autonomy_light/heartbeat .

- 내부 domain의 autonomy-light node에서 발행된다.
- 상태는 waiting_for_lidar , degraded:waiting_for_point_lio_odom , degraded:waiting_for_point_lio_map , ready:... 형태다.

7. LiDAR Network

livox_interface

MID360이 연결된 유선 NIC 이름이다. 예: enP8p1s0 .

livox_host_ip

Jetson host IP/CIDR이다. 예: 192.168.1.50/24 .

livox_lidar_ip

MID360 LiDAR IP다. 예: 192.168.1.166 .

- 틀리면 driver는 뜨더라도 point cloud가 안 나오거나 bind/config가 실패할 수 있다.

8. Publish 및 Missing Cell 처리

publish_rate_hz

height map, odom, TF 출력 주기다. 현재값: 50.0 .

- 높이면 반응은 빠르지만 외부 domain 부하가 증가한다.
- 제어용 SBC가 느려지면 30Hz 테스트를 권장한다.

interpolation_max_passes

비어있는 cell을 주변 valid cell 평균으로 채우는 반복 횟수다. 현재값: 2 .

- 키우면 구멍이 줄지만 장애물 잔상이 퍼질 수 있다.
- 장애물이 사라진 뒤 잔상이 있으면 줄인다.

interpolation_min_neighbors

interpolation에 필요한 주변 valid neighbor 수다. 현재값: 3 .

- 키우면 더 보수적으로 채운다.
- 낮추면 구멍은 줄지만 노이즈가 퍼질 수 있다.

fill_remaining_height

마지막까지 비어있는 cell에 넣을 기본 높이다. 현재값: 0.0 .

- 제거기가 unknown을 처리하지 못하면 0으로 채우는 것이 안전하다.
- unknown을 명시적으로 쓰고 싶으면 NaN 유지 구조가 필요하다.

9. Backend 선택

algorithm.elevation_backend

height map cell 대표값을 고르는 backend다. 현재값: autonomy_min_z .

- autonomy_min_z : autonomy의 MinZElevationBackend 개념. cell 내부 낮은 표면을 우선 잡는다.
- min_z : autonomy_min_z alias.
- robust : 기존 autonomy-light 방식. 낮은 cluster 평균 또는 percentile을 사용한다.

권장:

- 바닥/장애물 제어용 height map: autonomy_min_z
- 낮은 outlier가 너무 많고 바닥이 거칠 때: robust

10. Clipping

algorithm.clipping.enabled

height map 최종값을 범위 안으로 자를지 여부다. 현재값: true .

algorithm.clipping.min_z

최소 height 값이다. 현재값: 0.0 .

- 바닥 아래로 튀는 음수 노이즈를 0으로 누른다.
- 작은 아래턱/내리막을 봐야 하면 -0.05 ~ -0.10 으로 낮춘다.

algorithm.clipping.max_z

최대 height 값이다. 현재값: 0.48 .

- 과도하게 높은 누적점이 제어 입력을 혼드는 것을 막는다.
- 높은 장애물 구분이 필요하면 올린다.
- `/autonomy_light/height_map_data` 의 `base_height` 로도 사용된다. 이 값을 바꾸면 평지 근처 출력값의 기준도 같이 바뀐다.
- 제거기가 0.48 기준으로 학습/튜닝되어 있으면 신중하게 변경한다.

11. Supported Min-Z

`algorithm.min_z.min_points_per_cell`

cell 대표값을 채택하기 위한 최소 point 수다. 현재값: 3 .

- 올리면 노이즈가 줄지만 장애물/얇은 물체 반응이 늦어진다.
- 낮추면 반응은 빠르지만 지지직거리는 단발 노이즈가 늘 수 있다.

`algorithm.min_z.supported_min_enabled`

단순 최저점 대신, 낮은 높이대에 충분한 support가 있는 cluster를 사용할지 여부다. 현재값: `true` .

`algorithm.min_z.support_band`

낮은 surface cluster로 묶을 z 폭이다. 단위는 meter. 현재값: 0.04 .

- 키우면 더 많은 점이 support로 묶여 안정적이다.
- 너무 키우면 바닥과 낮은 장애물이 섞인다.

12. Obstacle Override

`algorithm.min_z.obstacle_override_enabled`

cell 안에 바닥보다 충분히 높은 cluster가 있으면 장애물로 우선 선택할지 여부다. 현재값: `true` .

- supported min-z가 바닥만 계속 고르는 문제를 보완한다.

`algorithm.min_z.obstacle_min_height`

바닥 cluster보다 얼마나 높아야 장애물로 볼지 정하는 threshold다. 현재값: 0.06 .

- 낮추면 장애물 반응이 빨라진다.
- 너무 낮추면 바닥 노이즈를 장애물로 볼 수 있다.

`algorithm.min_z.obstacle_min_points`

장애물 cluster로 인정할 최소 point 수다. 현재값: 2 .

- 올리면 false positive가 줄지만 얇은 장애물 반응이 늦다.
- 낮추면 빠르지만 노이즈에 민감하다.

`algorithm.min_z.obstacle_support_band`

장애물 cluster로 묶을 z 폭이다. 현재값: 0.06 .

- 키우면 듬성한 장애물 point도 묶인다.
- 너무 키우면 서로 다른 높이의 점이 같은 장애물 cluster로 섞인다.

13. Cloud Registered Fill

`algorithm.cloud_registered_fill.enabled`

Point-LIO local map에서 빈 cell을 `/cloud_registered` 현재 scan으로 보조 채움할지 여부다. 현재값: `false` .

- `true`면 초기 구멍은 줄지만 raw scan 노이즈가 들어올 수 있다.

`algorithm.cloud_registered_fill.percentile`

보조 fill cell에서 사용할 percentile이다. 현재값: 0.15 .

`algorithm.cloud_registered_fill.min_points_per_cell`

보조 fill을 인정할 cell별 최소 point 수다.

`algorithm.cloud_registered_fill.initial_floor_fill_enabled`

초기 local map coverage가 낮을 때 바닥 추정값으로 빈 cell을 채울지 여부다.

`algorithm.cloud_registered_fill.initial_floor_max_local_coverage`

초기 floor fill을 허용할 최대 local coverage다.

`algorithm.cloud_registered_fill.floor_min_points`

registered cloud에서 floor를 추정하기 위한 최소 point 수다.

`algorithm.cloud_registered_fill.floor_support_band`

floor 후보 cluster를 묶는 z 폭이다.

`algorithm.cloud_registered_fill.initial_keep_min_support`

초기 floor fill 시 현재 cell 값을 유지하기 위한 최소 support count다.

14. Frame Aggregation

`algorithm.frame_aggregation.robust_height_gate`

현재 grid와 이전 grid의 height 차이가 이 값 이하이면 temporal merge를 허용한다. 현재값: 0.04 .

`algorithm.frame_aggregation.intra_cell_min_support_gap`

robust backend에서 낮은 cluster를 만들 때 쓰는 z 폭이다.

`algorithm.frame_aggregation.intra_cell_min_support_count`

robust backend에서 낮은 cluster 평균을 쓰기 위한 최소 support 수다.

`algorithm.frame_aggregation.edge_mix_height_diff`

이전 grid와 현재 grid를 섞을 수 있는 edge 차이 threshold다.

`algorithm.frame_aggregation.edge_prefer_prev_support_count`

현재 cell support가 이 값보다 작고 이전값과 차이가 큰 경우 이전값을 유지한다.

- 값이 크면 잔상이 늘 수 있다.
- 값이 작으면 새 관측으로 빨리 바뀐다.

`algorithm.frame_aggregation.cell_height_percentile`

robust backend에서 support가 부족할 때 사용할 percentile이다.

`algorithm.frame_aggregation.temporal_alpha`

현재값과 이전값을 섞는 비율이다. 현재값: 1.0 .

- 1.0 : 현재값만 사용한다. 반응이 빠르다.
- 낮출수록 smoothing은 강하지만 장애물 반응과 사라짐이 늦어진다.
- 장애물 잔상을 줄이고 싶으면 먼저 1.0 을 유지한다.

15. Isolated Filter

`algorithm.isolated_filter.radius`

isolated noise 판단에 사용할 주변 cell 반경이다. 현재값: 1 .

`algorithm.isolated_filter.min_support_neighbors`

비슷한 높이의 neighbor가 최소 몇 개 있어야 정상 cell로 볼지 정한다. 현재값: 3 .

- 올리면 지지직거리는 점이 줄어든다.
- 너무 높으면 새 장애물 edge가 지워질 수 있다.

`algorithm.isolated_filter.support_height_diff`

neighbor가 같은 surface로 인정되는 height 차이다. 현재값: 0.035 .

`algorithm.isolated_filter.outlier_height_diff`

cell 값이 주변 median과 이 값 이상 다르면 outlier로 보정한다. 현재값: 0.04 .

`algorithm.isolated_filter.every_n_frames`

isolated filter 적용 주기다. 현재값: 1 .

- 1이면 매 frame 적용한다.
- 키우면 계산량은 줄지만 노이즈가 더 보인다.

16. Hole Fill

`algorithm.hole_fill.radius`

빈 cell을 채울 때 볼 neighbor 반경이다. 현재값: 1 .

`algorithm.hole_fill.min_neighbors`

hole fill에 필요한 최소 neighbor 수다. 현재값: 4 .

`algorithm.hole_fill.max_height_diff`

neighbor들의 height 범위가 이 값보다 작을 때만 hole fill을 한다. 현재값: 0.015 .

- 키우면 구멍이 줄지만 edge가 번질 수 있다.
- 줄이면 edge는 보존되지만 구멍이 늘 수 있다.

17. Bilateral Filter

`algorithm.bilateral.radius`

bilateral smoothing 반경이다.

`algorithm.bilateral.sigma_spatial`

공간 거리 가중치 sigma다.

`algorithm.bilateral.sigma_height`

height 차이 가중치 sigma다.

`algorithm.bilateral.max_height_diff`

이 height 차이를 넘는 neighbor는 smoothing에 쓰지 않는다.

`algorithm.bilateral.passes`

bilateral filter 반복 횟수다. 현재값: 0 .

- 0이면 꺼짐.
- 노이즈가 남으면 1부터 테스트한다.
- edge가 무너지면 다시 0으로 둔다.

`algorithm.bilateral.every_n_frames`

bilateral filter 적용 주기다.

18. 실행 관련

`start_lidar_driver`

autonomy-light가 Livox driver를 child process로 시작할지 여부다.

- 실차 기본값은 true 다.
- 외부에서 Livox driver를 따로 실행 중이면 false 로 둘 수 있지만 topic과 domain을 맞춰야 한다.

`start_point_lio`

autonomy-light가 Point-LIO를 child process로 시작할지 여부다.

- 실차 기본값은 true 다.
- 외부 Point-LIO를 따로 실행 중이면 false 로 둘 수 있지만 `/aft_mapped_to_init` , `/point_lio/local_map` 입력이 내부 domain에 있어야 한다.

`child_use_sim_time`

child process에 sim time을 사용할지 여부다.

- 실차에서는 false 를 유지한다.
- simulation mode에서만 true 로 쓴다.

`monitor_raw_lidar`

raw LiDAR topic 수신 여부를 heartbeat 상태에 반영할지 정한다. 현재값: false .

- true 로 켜면 `/livox/lidar` 가 멈췄을 때 `waiting_for_lidar` 상태를 볼 수 있다.
- 약간의 구독 부하가 생기므로 기본 운용에서는 꺼둔다.

19. 추천 튜닝 프리셋

노이즈 억제 우선

```
algorithm:
  min_z:
    min_points_per_cell: 4
    support_band: 0.05
    obstacle_min_points: 3
  isolated_filter:
    min_support_neighbors: 4
  bilateral:
    passes: 1
```

- 장점: 지지직거림 감소.
- 단점: 얇은 장애물이나 빠른 장애물 반응이 늦을 수 있다.

장애물 반응 우선

```
algorithm:
  min_z:
    min_points_per_cell: 2
    obstacle_min_height: 0.04
    obstacle_min_points: 1
  isolated_filter:
    min_support_neighbors: 2
```

- 장점: 장애물이 빨리 올라온다.
- 단점: random spike가 늘 수 있다.

잔상 감소 우선

```
interpolation_max_passes: 1
algorithm:
  frame_aggregation:
    temporal_alpha: 1.0
  hole_fill:
    max_height_diff: 0.01
```

- 장점: 사라진 장애물이 덜 남는다.
- 단점: 구멍이 늘 수 있다.

20. 문제별 판단표

증상	먼저 볼 파라미터	조치
제어용 PC 렉	debug_local_map_ros_domain_id	internal_ros_domain_id 와 같게 둔다
바닥이 지지직거림	min_points_per_cell, support_band	point support를 늘린다
장애물 반응이 늦음	obstacle_min_height, obstacle_min_points	threshold를 낮춘다
장애물 잔상	interpolation_max_passes, hole_fill, previous merge	fill 계열을 줄인다
edge가 둔함	elevation_resolution, bilateral.passes, hole_fill.max_height_diff	resolution을 낮추거나 smoothing을 줄인다
구멍이 많음	hole_fill.min_neighbors, interpolation_max_passes	fill을 늘린다
낮은 outlier가 장애물처럼 보임	clipping.min_z, min_points_per_cell	clipping과 support를 강화한다